

Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение высшего
образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Информатика и системы управления»

КАФЕДРА «Программное обеспечение ЭВМ и информационные технологии»

Лабораторная работа № 6

по дисциплине «Анализ Алгоритмов»

Тема Методы решения задачи коммивояжёра

Студент Поляков А.И.

Группа ИУ7-52Б

Преподаватели Волкова Л.Л.

Москва, 2024

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
1 Аналитическая часть	5
1.1 Формализованная постановка задачи	5
1.2 Методы решения	5
1.2.1 Метод полного перебора	5
1.2.2 Метод муравьиной колонии	6
1.3 Вывод из аналитической части	7
2 Конструкторская часть	8
2.1 Требования к программному обеспечению	8
2.2 Алгоритм полного перебора	8
2.2.1 Метод генерации перестановок	8
2.2.2 Алгоритм	8
2.3 Муравьиный алгоритм	10
2.4 Вывод из конструкторской части	11
3 Технологическая часть	12
3.1 Средства разработки	12
3.2 Реализация алгоритмов	12
3.3 Оценка сложности алгоритмов	16
3.4 Тестирование	16
3.5 Вывод из технологической части	17
4 Исследовательская часть	18
4.1 Параметризация	18
4.1.1 Класс данных	18
4.1.2 Результаты параметризации	19
4.2 Вывод из исследовательской части	19
ЗАКЛЮЧЕНИЕ	20

ПРИЛОЖЕНИЕ А	21
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	22

ВВЕДЕНИЕ

Задача коммивояжёра является одной из известных задач комбинаторной оптимизации, заключающейся в нахождении оптимального замкнутого маршрута по заданным точкам. Решения этой задачи находят применение в транспортной логистике для оптимизации маршрутов доставки товаров [1], а также для планирования сложных путешествий через несколько мест. [2]

История задачи насчитывает много лет, так как впервые её рассмотрел Эйлер в 1736 году в контексте проблемы семи Кенигсбергских мостов, хотя формального описания и названия тогда не было [3]. Первое неформальное упоминание о задаче встречается в книге 1832 года «Коммивояжер – как он должен вести себя и что должен делать для того, чтобы доставлять товар и иметь успех в своих делах – советы старого курьера», где проблема была описана, но не была предложена формальная математическая модель для её решения. Первое формальное описание задачи коммивояжёра принадлежит Карлу Менгеру, который представил обобщённую версию этой задачи. [4]

Целью данной работы является рассмотрение методов решения задачи коммивояжёра.

Для достижения этой цели требуется решить следующие задачи:

- постановка задачи коммивояжёра;
- рассмотрение методов полного перебора и муравьиной колонии с элитными муравьями;
- разработка рассмотренных алгоритмов решения задачи;
- реализация разработанных алгоритмов;
- проведение параметризации для алгоритма на основе метода муравьиной колонии.

1 Аналитическая часть

1.1 Формализованная постановка задачи

Задача коммивояжёра может быть описана следующим образом: коммивояжёр, покидая определённый город, стремится посетить $n - 1$ других городов ровно один раз и вернуться обратно. Известно, что между любыми двумя городами можно перемещаться, и известны расстояния между ними. Необходимо определить порядок посещения городов, чтобы минимизировать общее пройденное расстояние. [5]

Это определение можно формализовать с помощью теории графов, вводя некоторые термины и определения. Неориентированный граф (неорграф) G представляет собой пару (V_n, E_n) , где V_n — конечное множество, состоящее из вершин, а E_n — множество неупорядоченных пар вершин, называемых рёбрами. [6]

Циклом в неорграфе G называется последовательность вершин $v_1, \dots, v_n$, где все v_i различны, за исключением v_1 и v_n , и все рёбра также различны. [6] Цикл, который проходит через каждую вершину графа, называется гамильтоновым циклом. [7]

Неорграф G считается полным, если для любых различных вершин v_i и v_j из V_n существует ребро в E_n , соединяющее их. Взвешенным неорграфом называется пара (G, F) , где F — функция, сопоставляющая каждому ребру графа некоторое действительное число, называемое весом ребра. Весом цикла называется сумма весов всех рёбер, входящих в него. [7]

Таким образом, обобщённая задача коммивояжёра формулируется следующим образом: пусть дан взвешенный полный неорграф G . Задача коммивояжёра заключается в поиске гамильтонова цикла в G с минимальным весом.

На рисунке 1.1 представлена формализованная постановка задачи в нотации IDEF0.

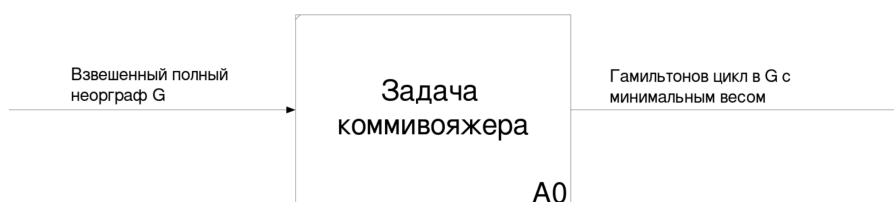


Рисунок 1.1 — Формализованная постановка задачи в нотации IDEF0

1.2 Методы решения

В данной работе будут рассмотрены два метода решения задачи коммивояжёра: метод полного перебора и метод муравьиной колонии.

1.2.1 Метод полного перебора

Метод полного перебора подразумевает решение задачи, при котором формируются все возможные гамильтоновы циклы, просчитывается их длина и выбирается тот, у которого эта

длина минимальна. Однако с увеличением числа узлов в графе сложность метода существенно возрастает, так как для графа из n узлов существует $\frac{(n-1)!}{2}$ гамильтоновых циклов. [9]

1.2.2 Метод муравьиной колонии

Метод муравьиной колонии – один из стохастических методов, применяющихся для решения задач комбинаторной оптимизации, в частности для решения задачи коммивояжёра. Метод основан на способе обмена информацией у муравьев, при котором они передают информацию о привлекательности путей с помощью меток (феромонов). [10].

Конструирование путей муравьёв

На данном этапе каждый из m муравьёв инициализируется с циклом с единственным начальным узлом и на каждом шаге стохастически выбирает узел из тех, которые он не посещал, и добавляет в свой цикл. Выбор узла на очередном этапе цикла производится с вероятностью, вычисляемой по формуле:

$$p(e_j^i) = \frac{\tau_{ij}^\alpha \cdot f(e_j^i)^\beta}{\sum \tau_{ij}^\alpha \cdot f(e_j^i)^\beta}, \quad (1.1)$$

где τ_{ij} - количество феромона на ребре между i -м и j -м узлами,
 $f(e_j^i)$ - функция выражающая «привлекательность» ребра между i -м и j -м узлами для муравья.

Обновление феромонов

На данном этапе формируются положительные и отрицательные обратные связи. Каждый муравей проходя по своему пути распыляет на нём феромоны, при этом чем длиннее путь, тем меньше распылённое количество феромонов. Распылённое число феромонов для k -го муравья рассчитывается по формуле:

$$\Delta\tau_{ij}^k = \begin{cases} \frac{Q}{L_k}, & \text{если ребро (i, j) есть в цикле муравья,} \\ 0, & \text{иначе,} \end{cases}, \quad (1.2)$$

где L_k - длина цикла k -го муравья, Q - константа.

Для того, чтобы количество феромона не было решающим фактором после нескольких циклов в модели есть испарение феромона, которое в конце цикла уменьшает концентрацию феромона на каждом ребре.

Элитные муравьи

Элитные муравьи – модификация муравьиного алгоритма. Элитные муравьи на каждой итерации идут по лучшему найденному пути и распыляют по нему феромоны, таким образом увеличивая привлекательность его частей для других муравьёв. С данной модификацией расчёт

феромона на каждой итерации происходит по формуле:

$$\tau_{ij} = (1 - p)\tau_{ij} + \sum_{k=0}^m \Delta\tau_{ij}^k + me \cdot \frac{Q}{L_b}, \quad (1.3)$$

где me - количество элитных муравьёв, L_b - длина лучшего цикла.

1.3 Вывод из аналитической части

В результате аналитического раздела была рассмотрена формальная постановка задачи коммивояжёра, а также рассмотрены два метода её решения.

2 Конструкторская часть

2.1 Требования к программному обеспечению

К разрабатываемой программе предъявлен ряд требований:

Входные данные: Взвешенный неориентированный граф, заданный матрицей стоимостей.

Выходные данные: Оптимальный гамильтонов цикл, в случае метода полного перебора, субоптимальный гамильтонов цикл в случае муравьиного алгоритма.

2.2 Алгоритм полного перебора

2.2.1 Метод генерации перестановок

Основной частью алгоритма полного перебора является алгоритм генерации перестановок, так как гамильтонов путь, как и гамильтонов цикл, являются перестановками множества узлов графа. Существуют множество методов генерации перестановок [11] как рекурсивных, так и итеративных. В данной работе будет использоваться нерекурсивный метод Хипа (Heap's method) разработанный в [12].

2.2.2 Алгоритм

На рисунке 2.1 представлен алгоритм полного перебора для решения задачи коммивояжёра.

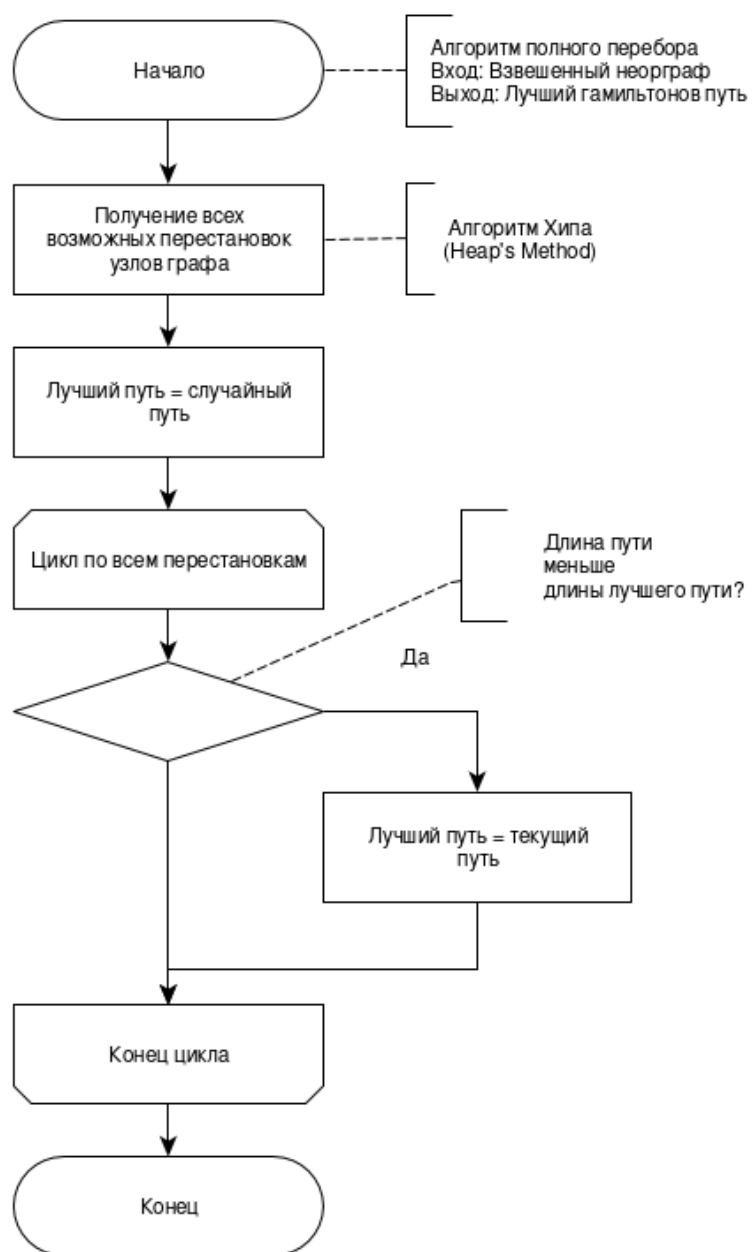


Рисунок 2.1 — Схема алгоритма полного перебора

2.3 Муравьиный алгоритм

На рисунках 2.2 и 2.3 представлен алгоритм муравьиного алгоритма для неорграфа с элитными муравьями.

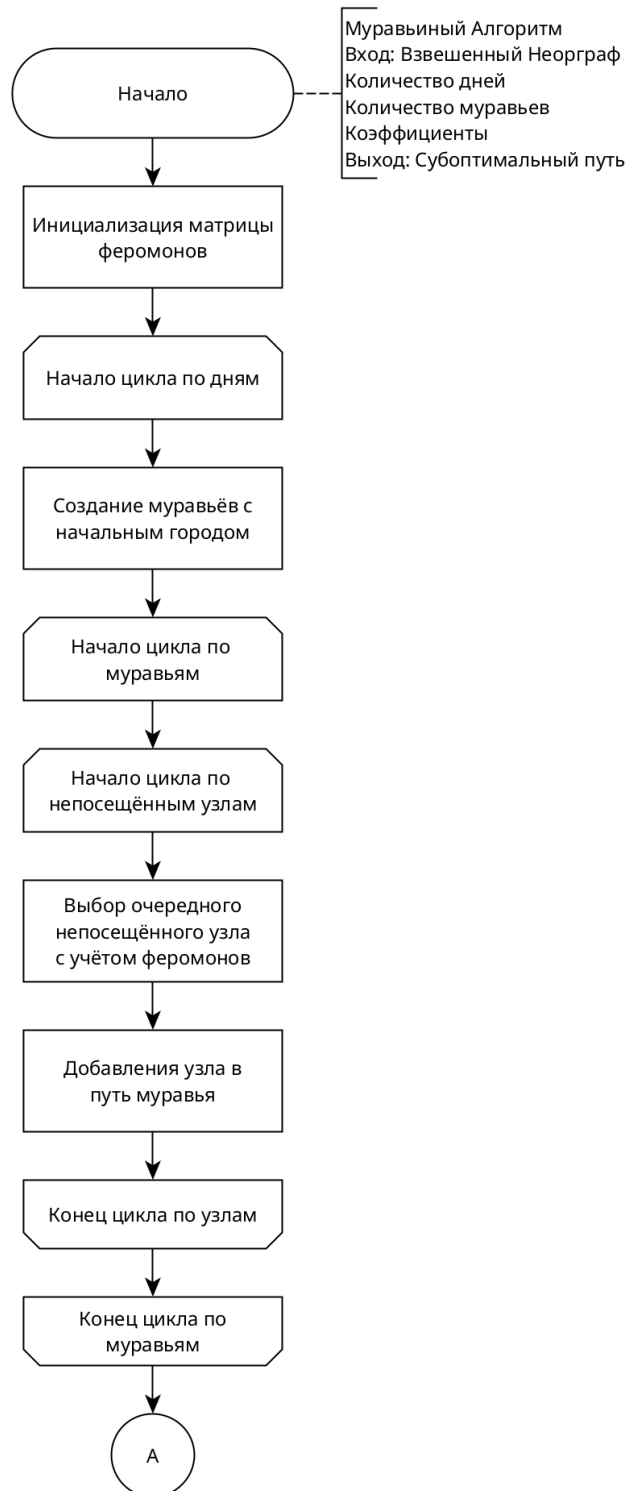


Рисунок 2.2 — Схема муравьиного алгоритма (начало)

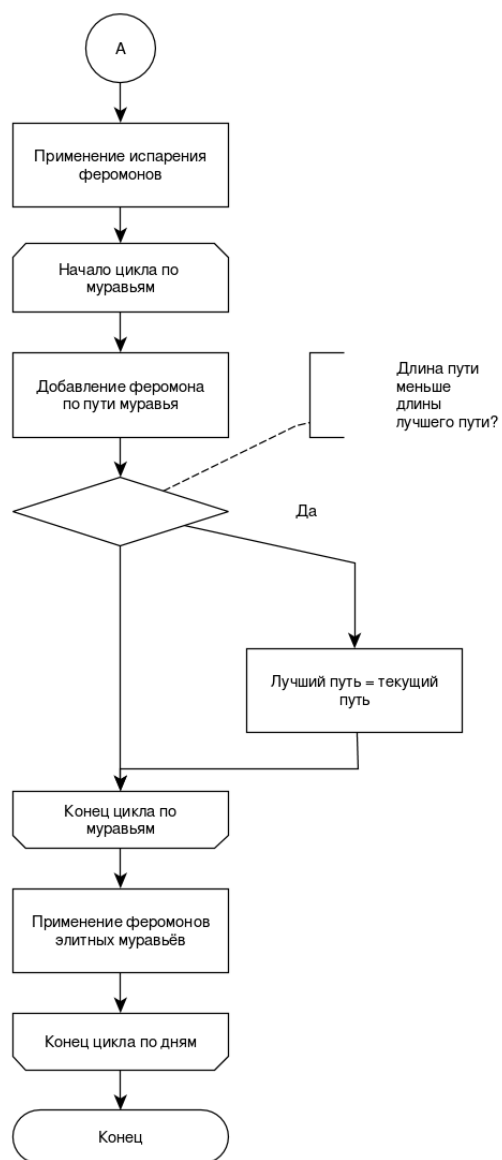


Рисунок 2.3 — Схема муравьиного алгоритма (конец)

2.4 Вывод из конструкторской части

В результате конструкторской части были определены требования к ПО, а также разработаны схемы алгоритмов полного перебора и муравьиного алгоритма.

3 Технологическая часть

3.1 Средства разработки

В качестве языка программирования был выбран python, так как данный язык обладает достаточными средствами для реализации алгоритмов.

3.2 Реализация алгоритмов

В листинге 3.1 представлен алгоритма полного перебора.

Листинг 3.1 — Алгоритм полного перебора

```
def calculate_distance(route, distance_matrix):
    total_distance = 0
    for i in range(len(route) - 1):
        total_distance += distance_matrix[route[i]][route[i + 1]]
    total_distance += distance_matrix[route[-1]][route[0]]
    return total_distance

def find_routes(current_city, visited, route, distance_matrix,
all_routes):
    if len(visited) == len(distance_matrix):
        all_routes.append(route[:])
        return

    for next_city in range(len(distance_matrix)):
        if next_city not in visited:
            visited.add(next_city)
            route.append(next_city)
            find_routes(next_city, visited, route, distance_matrix,
all_routes)
            route.pop()
            visited.remove(next_city)

def full_search(distance_matrix):
    n = len(distance_matrix)
    all_routes = []
    visited = set()

    visited.add(0)
    find_routes(0, visited, [0], distance_matrix, all_routes)
```

```

min_distance = float('inf')
best_route = None

for route in all_routes:
    current_distance = calculate_distance(route, distance_matrix)
    if current_distance < min_distance:
        min_distance = current_distance
        best_route = route

return best_route, min_distance

```

В листинге 3.2 представлена реализация муравьиного алгоритма.

Листинг 3.2 — Муравьиный алгоритм

```

import numpy as np
import random

class AntColony:
    def __init__(self, distance_matrix, num_ants, num_iterations, alpha,
        , beta, evaporation_rate, elite_factor, num_elite_ants):
        self.distance_matrix = distance_matrix
        self.num_ants = num_ants
        self.num_iterations = num_iterations
        self.alpha = alpha
        self.beta = beta
        self.evaporation_rate = evaporation_rate
        self.elite_factor = elite_factor
        self.num_elite_ants = num_elite_ants
        self.num_cities = len(distance_matrix)
        self.pheromone_matrix = np.ones((self.num_cities, self.
            num_cities))

    def run(self):
        best_route = None
        best_distance = float('inf')

        for _ in range(self.num_iterations):
            all_routes = []
            all_distances = []

            for _ in range(self.num_ants):
                route = self.construct_route()

```

```

        distance = self.calculate_distance(route)
        all_routes.append(route)
        all_distances.append(distance)

        if distance < best_distance:
            best_distance = distance
            best_route = route

        sorted_indices = np.argsort(all_distances)
        elite_routes = [all_routes[i] for i in sorted_indices[:self
            .num_elite_ants]]
        elite_distances = [all_distances[i] for i in sorted_indices
            [:self.num_elite_ants]]

        self.update_pheromones(all_routes, all_distances,
            elite_routes, elite_distances, best_route, best_distance
        )

    return best_route, best_distance

def construct_route(self):
    route = []
    visited = set()
    current_city = random.randint(0, self.num_cities - 1)
    route.append(current_city)
    visited.add(current_city)

    for _ in range(self.num_cities - 1):
        next_city = self.select_next_city(current_city, visited)
        route.append(next_city)
        visited.add(next_city)
        current_city = next_city

    return route

def select_next_city(self, current_city, visited):
    probabilities = []
    total_pheromone = 0.0

    for city in range(self.num_cities):
        if city not in visited:

```

```

        pheromone = self.pheromone_matrix[current_city][city]
        ** self.alpha
        distance = self.distance_matrix[current_city][city]
        probability = pheromone / (distance ** self.beta)
        probabilities.append(probability)
        total_pheromone += probability
    else:
        probabilities.append(0)

probabilities = [p / total_pheromone for p in probabilities]
return np.random.choice(range(self.num_cities), p=probabilities
    )

def calculate_distance(self, route):
    total_distance = 0
    for i in range(len(route) - 1):
        total_distance += self.distance_matrix[route[i]][route[i +
            1]]
    total_distance += self.distance_matrix[route[-1]][route[0]]
    return total_distance

def update_pheromones(self, all_routes, all_distances, elite_routes
    , elite_distances, best_route, best_distance):
    self.pheromone_matrix *= (1 - self.evaporation_rate)

    for route, distance in zip(all_routes, all_distances):
        pheromone_deposit = 1.0 / distance
        for i in range(len(route) - 1):
            self.pheromone_matrix[route[i]][route[i + 1]] +=
                pheromone_deposit
        self.pheromone_matrix[route[-1]][route[0]] +=
            pheromone_deposit

    for elite_route, elite_distance in zip(elite_routes,
        elite_distances):
        elite_pheromone_deposit = self.elite_factor /
            elite_distance
        for i in range(len(elite_route) - 1):
            self.pheromone_matrix[elite_route[i]]
                [elite_route[i + 1]] += elite_pheromone_deposit
        self.pheromone_matrix[elite_route[-1]][elite_route[0]]

```

3.3 Оценка сложности алгоритмов

Для алгоритма полного перебора оценка сложности составляет $O(n! * n) = O(n!)$, так как сложность алгоритма генерации перестановок $O(n!)$ [12], а сложность поиска цены пути составляет $O(n)$.

У муравьиного алгоритма оценка сложности рассчитывается следующим образом

$$O(d * (m * (n^2) + (n^2) + m * n + n)) = O(dmn^2), \quad (3.1)$$

где

- d – количество дней;
- m – число муравьёв;
- n – число узлов.

3.4 Тестирование

Тестирование алгоритма полного перебора возможно провести с проверкой точного значения, однако для муравьиного алгоритма можно проверить лишь то, что результирующий цикл не содержит все узлы, то есть является гамильтоновым циклом.

Тест 1

Входные данные:

$$\begin{bmatrix} 0 & 10 & 15 & 20 \\ 10 & 0 & 35 & 25 \\ 15 & 35 & 0 & 30 \\ 20 & 25 & 30 & 0 \end{bmatrix}$$

Ожидаемое значение:

Лучший маршрут: [0, 1, 3, 2]

Минимальное расстояние: 80

Выходные данные полного перебора:

Лучший маршрут: [0, 1, 3, 2]

Минимальное расстояние: 80

Выходные данные муравьиного алгоритма:

Лучший маршрут: [3, 2, 0, 1]

Минимальное расстояние: 80

Тест 2

Входные данные:

0	29	20	21	16	31	100	12	4	31
29	0	15	17	28	40	72	21	29	41
20	15	0	28	12	25	81	18	24	30
21	17	28	0	23	39	63	12	18	25
16	28	12	23	0	30	75	14	21	27
31	40	25	39	30	0	50	30	35	45
100	72	81	63	75	50	0	90	100	80
12	21	18	12	14	30	90	0	10	15
4	29	24	18	21	35	100	10	0	20
31	41	30	25	27	45	80	15	20	0

Ожидаемое значение:

Лучший маршрут: [0, 5, 6, 3, 1, 2, 4, 7, 9, 8]

Минимальное расстояние: 241

Выходные данные полного перебора:

Лучший маршрут: [0, 5, 6, 3, 1, 2, 4, 7, 9, 8]

Минимальное расстояние: 241

Выходные данные муравьиного алгоритма:

Лучший маршрут: [0, 8, 9, 7, 4, 2, 1, 3, 6, 5]

Минимальное расстояние: 241

3.5 Вывод из технологической части

В ходе технологической части работы были разработаны муравьиный алгоритм и алгоритм полного перебора для задачи коммивояжёра, а также проведена оценка их временной сложности. Все тесты успешно пройдены.

4 Исследовательская часть

4.1 Параметризация

Параметризация проводится по 3-м параметрам муравьиного алгоритма: α – коэффициент влияние феромонов на выбор муравья, p – коэффициент испарения феромонов, дни – количество дней в муравьином алгоритме.

4.1.1 Класс данных

В качестве класса данных для параметризации используем 3 графа, построенных на морских портах. В качестве весов графа использовались время путешествия между этими портами в днях, взятое из интернет-ресурса route.vesselfinder.com.

Граф 1:

- 1) Rotterdam;
- 2) Singapore;
- 3) A Coruna;
- 4) Abercastle;
- 5) Quincy;
- 6) Agropoli;
- 7) Aigiali Amorgou;
- 8) Lulsdorf;
- 9) Dortyol;
- 10) Heerewaarden.

Граф 2:

- 1) Dresden;
- 2) Kostrena;
- 3) Marsden Point;
- 4) Felixstowe;
- 5) Bay Roberts;
- 6) Hopewell;
- 7) Muasdale;
- 8) Carradale;
- 9) Orcadas;
- 10) Kusadasi.

Граф 3:

- 1) Al Qantarah;
- 2) Kwaadmechelen;
- 3) Sada;
- 4) Hadsund;

- 5) Alexandroupolis;
- 6) SAXEMARA;
- 7) Stokmarknes;
- 8) Marsaxlokk;
- 9) Miedzyzdroje;
- 10) Hoofdplaat.

4.1.2 Результаты параметризации

В результате параметризации оказалось, что самыми лучшими параметрами по заданному классу данных оказались при $\alpha = 0.3$, $p = 0.3$ и количеством дней равным 100. При этом данные параметры дают лучший результат как по средним параметрам, так и по минимальным значениям. Результаты параметризации для данных параметра представлены в таблице 4.1, а вся таблица параметризации представлена в приложении А.

Таблица 4.1 — Результаты параметризации муравьиного алгоритма при $\alpha = 0.3$, $p = 0.3$, дни = 100

min1	max1	mean1	min2	max2	mean2	min3	max3	mean3
0	0	0	0	0	0	0	0	0

4.2 Вывод из исследовательской части

В результате исследовательской части была проведена параметризация по заданному классу данных и выявлены лучшие параметры.

ЗАКЛЮЧЕНИЕ

Цель – рассмотрение методов решения задачи коммивояжёра – была выполнена. При этом в ходе работы были выполнены следующие задачи:

- сформулирована задача коммивояжёра;
- рассмотрены методы полного перебора и муравьиной колонии с элитными муравьями;
- разработаны рассмотренные алгоритмы решения задачи;
- реализованы разработанные алгоритмы;
- проведена параметризация для алгоритма на основе метода муравьиной колонии.

В результате работы были проанализированы сложности алгоритмов муравьиной колонии и полного перебора, проведена параметризация муравьиного алгоритма и выявлены лучшие параметры для класса данных, основанных на морских портах, а именно: $\alpha = 0.3$, $p = 0.3$ и количество дней – 100.

ПРИЛОЖЕНИЕ А

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. Володина Е.В., Студентова Е.А. Практическое применение алгоритма решения задачи коммивояжёра // Инженерный вестник Дона. - 2015. - №2.
2. Воронин Е.В., Бутузова Е.А. ПРИМЕНЕНИЕ ЗАДАЧИ КОММИВОЯЖЁРА // Символ науки. - Уфа: Символ науки, 2020. - С. 62-63.
3. Математический энциклопедический словарь.. - М.: Советская энциклопедия, 1988. - 848 с.
4. Alexander Schrijver. On the history of combinatorial optimization (till 1960) // 2005
5. Мудров В.И. Задача о Коммивояжёре. - М.: Знание, 1969. - 64 с.
6. Белоусов А.И., Ткачев С.Б. Дискретная математика: учебник для вузов. - 7-е изд. - М.: Издательство МГТУ им. Н. Э. Баумана, 2021. - 703 с.
7. Асанов М.О., Ткачев С.Б. Дискретная математика: графы, матроиды, алгоритмы : учебное пособие. - 3-е изд. - СПб.: Лань, 2020. - 364 с
8. Задача коммивояжёра // Большая Российская Энциклопедия URL: <https://bigenc.ru/c/zadachakommivoiazhera-tsp-61952a> (дата обращения: 09.12.2024).
9. Michael Junger, Gerhard Reinelt, Giovanni Rinaldi. The Traveling Salesman Problem. - Institut fur Informatik UNIVERSITAT ZU KOLN, 1994. - 112 с.
10. Marco Dorigo, Thomas Stützle. Ant Colony Optimization: Overview and Recent Advances // Handbook of Metaheuristics. - Switzerland: Springer International Publishin, 2019. - С. 311-351.
11. Robert Sedgwick. Permutations generation methods // Computing Survey. - 1977
12. Robert Sedgwick. Permutations generation methods // Princeton University. - 2002